



X4Pro - universal, fully relational EXFOR database

by V.Zerkin, IAEA:2021-2023, version 2026-08-28

This page presents progress in development X4Pro database and tools ([download](#)).

Concept.
X4Pro extends EXFOR-Relational database with experimental data points in original and computational form and additional information for data renormalization.

- The X4Pro database provides:
- access to all EXFOR meta-data and numerical data using only SQL commands, i.e. no need in original EXFOR, EXFOR parser and converter
 - data for renormalization: old and new monitor cross sections, old and new decay data (gamma lines intensity of measured reaction and monitor product)
 - experts' instructions for EXFOR data modifications
 - EXFOR Subentries and Datasets in X5-json format (since 2023-03-30)
 - evaluators' flags and scores (since 2025-01-10)

- Tasks of the project:
1. Desing of database schema
 2. Work out data and programming needs on typical examples
 3. Develop new concept of EXFOR data corrections and define workflow
 4. Envent/find out possible technical solutions for existing tasks, study problems, implement solutions
 5. Prepare for test/demo examples for end-user, confirm working tools on platfroms/enviromnent:
 - a) X4Pro: local SQLite database and remote MariaDB database
 - b) python on Windows, Linux, MacOS (plot with Plotly and Mathplotlib)
 - c) gfortran, gcc on Windows, Linux, MacOS
 - d) retrieve ENDF data from Web server and plot together with EXFOR data
 - e) populate NoSQL database (CouchDB) using X5-json from X4Pro as data source

Distribution (see [readme](#), [license](#), [copyright](#)), ver.2026-08-28:

- X4Pro/SQLite database, ver.2026-08-28
- Python and Fortran demo-programs, ver.2026-08-28
- Bash-scripts running demo-programs (CLI [\[1\]](#) and TUI [\[2\]\[3\]](#)), database statistics, ver.2026-08-28
- consists of 6 parts:

#	Dir	Description	SF:MF	Notes	Demo plot/output [code][modules]	Comment
Part-1. Retrieve data from local EXFOR database Python. Retrieve experimental data from X4Pro database and plot data using Plotly amd Matplotlib						
0.	part1-0-sig	Cross sections	SIG	AI-27(n,a):CS(Ei)	AI27na-1.html/json [code] AI27na-2.html/json [code]	Code: single .py file for copy/paste 2) data grouped Reaction-codes

1.	part1-1-sig	Cross sections	SIG	Al-27(n,a):CS(Ei)	Al27na-1.html/json [code][rx4db] Al27na-2.html/json [code]	Coding: modular (several .py files) 2) data grouped Reaction-codes
2.	part1-2-da	Angular distributions	DA	O-16(a,el):DA(Ei)	da0an.html/pdf/json [code][rx5da]	Plotly/matplotlib
3.	part1-3-dap	Angular distributions (partial)	PAR,DA	Li-6(he3,p):DA(θ)	dap0ei.html/pdf/json [code][rx4dap_ei]	Plotly/matplotlib
4.	part1-4-de	Emission spectra	DE	Th-232(n,xn):DE(Eo)	de0eo.html/pdf/json [code][rx5de]	Plotly/matplotlib
5.	part1-5-dae	Double differntial cross sections	DA/DE	F-19(n,xn):DAE(Eo)	dae0eo.html/pdf/json [code][rx4dae_e2]	Plotly/matplotlib
6.	part1-6-fy	Fission yeild (mass distribution)	FY	U-238(n,f):FY(A) Ei14MeV U-235(n,f):hist2(A,TKE)	U238nf-fy.html/pdf/json [code][rx4fya] me0x.html [code] me0mpl3d.pdf [code] me0mpl2d2.pdf [code]	1) Plotly/matplotlib 2) Plotly: Contour-2D 3) Matplotlib: plot_surface-3D 4) Matplotlib-2d, 2 subplots
7.	part1-7-covar	Covariance data stored in EXFOR	SIG	1) ²⁴¹ Am(n,2n) correlations ERR-T,MONIT-ERR,ERR-5 2) Reaction-reaction correlations	covar1.tto.html/pdf/json [code] covar2.tto.html/pdf/json [code]	1) Matplotlib-2d, 3 subplots 2) Matplotlib-2d, 1 subplot
8.	part1-8-reac 2026	Any type of EXFOR data	SIG, DA DE, DAE CSP, DAP TKE, ETA NUDAT FY	1) Partial cross section CSP 2) Temperatire dependent CST 3) Total kinetic energy TKE 4) Neutron yield ETA 5) Neutron yield NUBAR 6) DATA and DATA-MIN 7) Search by author etc.	1) CSP: sig1p.html/png/json, tt-out 2) CST: cst.html/png/json, tt-out 3) TKE: tke1.html/png/json, tt-out 4) ETA: eta1.html/png/json, tt-out 5) NUBAR: nu1.html/png/json, tt-out 6) DATA-MIN: si28nn.html/png/json, tto 7) Author1: Kokkoris.html/png/json, tto - [reac1.py] main code - [reac1help.py] display help-text - [reac1subr.py] retrieval subroutines	1) parameters: reaction-code, x-variable, filters 2) descriptions, x-variables for grouping are taken from the database ---under development---
	./	Common modules:			1) [dbConn.py] database connectivity with SQLite 2) [sqlsubr.py] implementation of SQL operations 3) [x4out.py] output datasets prepared for plotting (now: to JSON) 4) [exfor2plot-plotly.py] EXFOR plotting with Plotly 5) [exfor2plot-matpl.py] EXFOR plotting with Matplotlib	
Fortran. Retrieve X4Pro data from Fortan program via C subroutines						
8.	part1-fortran/	0) Simple example Fortran executing SQL 1) Double differntial cross sections 2) Cross sections 3) Cross sections	-- DA/DE SIG SIG	First 8 Entries from F-Area F-19(n,xn):DAE(Eo) Mn-55(n,a):CS(Ei) Mn-55(n,a):CS(Ei) →C4	f1.tto.html [code] dae1e2.tto.html [code] sig1.tto.html [code] sig1toc4.tto, C4 [code]	No plots, output text only
9.	part1-fortran/ See also implemen- tation in Python: part3-7-legrs2da	4) Retrieve LEG/RS and SIG from the same Entry; calculate missing Legendre coefficient L[0]; calculate angular distributions	LEG/RS SIG	Cu-0(n,el):DA,,LEG/RS(Ei) SIG→L[0] L[0..n],θ→DA(θ)	legrs2da.tto.html, C4, ZVD [code] terminal:output	FIND: SIG from <Entry, Reaction, Ei> CALC: SIG/4π → L[0]; CALC(Ei): L[0..n],θ → DA(Ei,θ)
	part1-fortran/	Common subroutines:			c1) [sql1sub.c] : fortran-to-sqlite3 adapter (by V.Zerkin) c2) [sqlite3.c][sqlite3.h] sqlite3 amalgamation (by SQLite team)	
Part-2. Retrieve local EXFOR and remote ENDF data						

Python. Retrieve and plot experimental (local EXFOR) and evaluated data (Web ENDF-API)						
1.	part2-1-sig1	Cross sections	SIG	Al-27(n,a):CS(Ei)	Al27na.html/pdf/json [code][rx4db] Al27na2.html/json	1) using Plotly and Matplotlib 2) data grouped Reaction-codes
2.	part2-2-da1an	Angular distributions (Ei:fixed)	DA	Mn-55(a,el):DA(θ)	da1an.html/pdf/json [code][rx5da]	DA + MF4/MF34 <i>#new</i>
3.	part2-3-da1ei	Angular distributions (Angle:fixed)	DA	C-12(a,el):DA(Ei)	da1ei.html/pdf/json [code][rx4da_ei]	Plotly/matplotlib
4.	part2-4-de1eo	Emission spectra	DE	Th-232(n,xn):DE(Eo)	de1eo.html/pdf/json [code][rx5de]	Plotly/matplotlib
5.	part2-5-dae1eo	Double differential cross sections	DA/DE	F-19(n,xn):DAE(Eo)	dae1eo.html/pdf/json [code][rx4dae_e2]	Plotly/matplotlib
6.	part2-6-fy	Fission yeild (mass distribution)	FY	U-238(n,f):FY(A) Ei14MeV	U238nf-fy.html/pdf/json [code][rx4fya][rweb1fy]	Plotly/matplotlib
7.	part2-7-fye 2024	Fission yeild (energy distribution)	FY	U-238(n,f):FY(E)	u235-nf-cd115g-cumFY.htm/pdf/json [code][rx4fye][e4subr]	Plotly/matplotlib
8.	part2-8-sig1par 2026	Partial cross section	CSP	Li-7(n,inl)Li-7,par,sig Li-7(n,n'):MF3.MT51	data: Li7ninl.html/png/json, tt-out - [sig1par.py] main code - [rx4sig1par.py] retrieval subroutines	---in progress---
9.	part2-9-nubar 2026	Neutron yield (nu-bar)	NU	U-235(n,f),[pr dl],nu MT452/455/456:MF1+MF31	PR-data: U235nf.html/png/json,ex,ev,tto DL-data: U235nf.html/png/json,ex,ev,tto TOT-data: U235nf.html/png/json,ex,ev,tto - [nubar.py] main code - [rx4nubar.py] retrieval subroutines	---in progress---
Additional common modules:					6) [websubr.py] GET: download text from NDS web-site 7) [rweb11.py] ENDF Web retrieval (SIG) 8) [rweb12.py] ENDF Web retrieval (DA, DE, DAE) 9) [endf2plot-plotly.py] ENDF plotting with Plotly 10) [endf2plot-matpl.py] ENDF plotting with Matplotlib	
Part-3. EXFOR data corrections and recalculations (for experts)						
Python. EXFOR data corrections (automatic*, users, experts*) and calculations + plot with ENDF data						
*Note. All input for corrections (monitor CS data, decay data, experts' operations) are stored in the database.						
1.	part3-1-auto1	Automatically renormalize CS using old monitor CS and new standards	SIG	Mn-55(n,a):CS(Ei) mon/std from database	auto1.html/pdf [code][mn55na_sub] orig.-retrieved: mn55na-0pt.json auto-corrected: mn55na-1pt.json	AUTO[0] [0]: MONITOR, MONIT-REF, MONIT,..
2.	part3-2-user1	Correct data by user's program	Ratio	pu239/u235(n,f):(Ei)	user1.html/pdf [code][myCorr1] [pu239nf203sql][pu239nf203corr]	USER
3.	part3-3-expert1	Apply experts corrections stored in X4Pro database	SIG	Zn-64(n,p):CS(Ei)	expert1.html/pdf [code][corr_subr] [x13597002sub] ;terminal:output orig.-retrieved: orig.json corrected-by-expert-db: corr.json	EXPERT
4.	part3-4-auto2	Automatically renormalize CS using old monitor CS and new standards, old and new decay data	SIG	V-51(n,p):CS(Ei)	auto2.html/pdf [code][x12898sig_sub] orig.-retrieved: orig.json auto-corrected: auto.json	AUTO[0][1][2] [1]: DECAY-DATA [2]: DECAY-MON

5.	part3-5-ratio2sig	Retrieve ratios, calculate CS and uncertainties using latest standards	Ratio	V-51(n,p)/U-238(n,f):(Ei) Ratio→SIG	ratio2cs.html/pdf [code][x12898sig_sub] orig.-retrieved: ratio.json auto-corrected: sig.json	CALC: $\text{Ratio} \times U\text{-}238(n,f) \rightarrow \text{SIG}$
6.	part3-6-daleg2sig	Retrieve Legendre coefficient L[0], calculate cross sections and errors	LEG	B-11(n,el):DA,,LEG(Ei) L[0]→SIG	B11nel.html/pdf/json [code][rx4db2] terminal:output	CALC: $L[0] \times 4\pi \rightarrow \text{SIG}$
7.	part3-7-legrs2da	Retrieve LEG/RS and SIG from the same Entry; calculate missing Legendre coefficient L[0]; calculate angular distributions	LEG/RS SIG	Cu-0(n,el):DA,,LEG/RS(Ei) SIG→L[0] $L[0..n], \theta \rightarrow \text{DA}(\theta)$	Cu0nel_da.html/pdf/json0/json1 [code][rx5da][rx5legrs] terminal:output	FIND: SIG from <Entry, Reaction, Ei> CALC: $\text{SIG}/4\pi \rightarrow L[0]$; CALC(Ei): $L[0..n], \theta \rightarrow \text{DA}(Ei, \theta)$
8.	part3-8-leg2r33	Retrieve Legendre coefficient calculate angular distributions store R33dat (draft of R33)	LEG	Li-6(p,he3):LEG(Ei)	leg2da.html/pdf/r33dat [code][rx4leg_ei][r33out]	CALC(Ei): $L[0..n], \theta \rightarrow \text{DA}(Ei, \theta)$
Additional common modules:					11) [auto_corr.py] automatic correction of EXFOR data 12) [expert_corr.py] expert's correction from X4Pro	

Part-4. Export contents of X4Pro to NoSQL databases (for NoSQL databases developers)

Python. Retrieve data from X4Pro and store them as JSON in NoSQL-Document database

**Note. Since 2022-08-30 X4Pro includes two tables presenting Subentries and Datasets in X5-json format.*

***Note. Since 2024-09-04 these two tables are included from X4Pro distribution on request only.*

****Note. X5-json is available on GitHub: <https://github.com/vzerkin/EXFOR-X5json>*

1.	part4-0-couchdb	Retrieve JSON presentation of Subent in X5-json format from Table x4pro_x4z, created a Dictionary "Entry" and store it as Document in CouchDB database	-NA-	JSON files for each Subent are retrieved row-by-row and collected into Python List	program: [code] Retrieved from X4Pro: --subent.001 10010001.json.txt --subent.002 10010002.json.txt stored: 10010.json in CouchDB --entries: list of documents in CouchDB --entry: 10010.json in CouchDB	[howto]
----	-----------------	--	------	--	--	-------------------------

Part-5. Search for mistakes in EXFOR library using X4Pro (for NRDC-EXFOR compilers)

python Retrieve data from X4Pro and store them as plain text, CSV and JSON files

sqlite3 Simple command-line program (shell) for SQLite

SQL Structured Query Language used as instructions for programs above.

1.	part5-0-sql 2024	1) NRDC-2024/A47: Analyse the zero values coded under the headings PARITY, ERR-T and DATA-ERR. 2) Text plotting DATA from a Dataset	-NA-	Find non-physical values of parity ($=0$) and uncertainties (<0)	program: [err-t.py] found: [csv] [json] program: [data-err.py] found: [csv] [json] program: [exec1sql.py] • inc.Energy ≤ 0 : [SQL] [CSV] [JSON] • Parity $=0$: [SQL] [CSV] [JSON] utility code: [sqlite3] script: [runme.sh] • inc.Energy ≤ 0 : [SQL] [-csv] [-box] • histogram: [SQL] result: [-header -box]	[howto]
----	----------------------------	--	------	--	---	-------------------------

Part-6. X4Pro for evaluators: ratios to cross sections + automatic renormalization + evaluators flags and scores

1.	part6-0-eval1flags	Summary of tables with evaluators' flags and scores	-NA-	Flags: accepted/rejected Scores:good/doubtful/outlier	Overview: [sql] [txt] Experts: [sql] [txt] IAEA-CRP review: [sql] [txt]	[howto] EXFOR-online [evaluators_flags]
----	--------------------	---	------	--	---	--

	2025				Stastical test: [sql] [txt] ---Save eval-score tables to CSV/JSON: program: [exec2sql1.py] select: [sql] result: [csv] , [json] select: [sql] result: [csv] , [json]	
9.	part6-1-eval2sig 2025	Retrieve CS, ratios, monitor data; convert ratios to cross sections, renormalize CS to new standards and new decay data. MyPlot: advanced interactive graphics.	SIG	(N,ABS),,ALF → SIG Ratio×std → SIG Zn-64(N,P):SIG → renorm.	Python: [code] [x4evalscore] [rx4dbr] • U235ng.html/pdf • Pu240nf.html/pdf • Al27na.html/pdf • Al27na2.html/pdf • zn64np.html/pdf MyPlot: [code] , [endf2plot_js.py] , [exfor2plot_js.py] • Pu240nf-myplot • Mn-55-myplot	CALC: ALF×SIG:(n,f) → SIG(n,g) CALC: (R1)/(R2)×std(R2) → SIG(R1) CALC: SIG/mon×std → SIG MyPlot: plotting stored JSON files with more interactions (JavaScript)

Created by [V.Zerkin](#), IAEA, 09-Dec-2021

Last updated: 08/28/2026 13:17:21